GOVERNMENT HOSPITAL EMERGENCY RESOURCE ALLOCATION & REFERRAL SYSTEM

A Pure Java Application Demonstrating Core OOP Principles, Advanced Data Structures & Dijkstra's Algorithm

Academic Project Presentation	**Core Language: Pure Java SE (No External Frameworks)**
Project Scope: Emergency Triage, Bed Allocation & Regional Referral	**Data Structures:** PriorityQueue, HashMap, Weighted Graph, ArrayList, Min-Heap
System Environment: Nalam District Health Network	**Routing Engine:** Dijkstra's Shortest Path Algorithm

1. Technology Stack & Architectural Foundation

Detailed explanation of core runtime, libraries, language features, and tooling

Layer / Component	Technology Selected	Role & Technical Justification in Project
Programming Language	**Java Standard Edition (Java SE)** OpenJDK 25 / Java 11+	• **Zero External Dependencies:** Built strictly with Java core packages (java.util.*, java.time.*). • **Single-Source Execution:** Leverages JEP 330 to run HospitalApp.java in-memory with 0 lingering class files. • **Strong Type Safety:** Eliminates runtime class errors in medical triage state management.
OOP Architecture	**Pure Object-Oriented Principles**	• **Encapsulation:** Private bed & doctor states with state guardrails. • **Polymorphism:** Abstract Bed specialized into GeneralBed & ICUBed. • **Abstraction:** AdmissionPolicy interface for decoupled clinical eligibility rules. • **Composition:** Hospital composes Bed/Doctor HashMaps; ReferralService composes RoutePlanner.
Data Structures	**Java Collections Framework**	• **PriorityQueue:** Min-heap ordering patients by Severity rank & arrival time. • **HashMap / LinkedHashMap:** O(1) instant entity lookups by ID. • **Weighted Graph:** Adjacency map Map<String, Map<String, Integer>>. • **ArrayList:** Amortized O(1) chronological transaction audit log.
Algorithmic Engine	**Dijkstra's Algorithm + Min-Heap**	• **O((V + E) log V)** shortest road route pathfinding between hospitals. • **Greedy Resource Locking:** Immediate two-resource reservation upon selection.
Build & Run Tooling	**Windows Native Batch Scripts**	• **compile.bat:** Verifies syntax and compilation integrity in temporary memory. • **run.bat:** Launches the interactive system directly in-memory. • **test.bat:** Executes automated verification tests with 100% test pass rate.

2. Problem Analysis & Core Allocation Rules

Clinical assumptions, emergency triage constraints, and multi-tier hospital referral principles

Problem Analysis & Constraints

- **Emergency Influx:** District hospitals face simultaneous patient surges with varying clinical urgency.
- **Resource Contention:** Critical ICU (ventilator) and General ward beds are scarce and cannot be over-allocated.
- **Staffing Constraints:** Doctors work specific shifts and specialize (Critical Care vs General). An available bed is useless without an on-duty qualified doctor.
- **Inter-Hospital Transfer:** When local capacity is full, patient transfer must be routed to the fastest reachable hospital.

Core Allocation & Processing Rules

- **Severity Hierarchy:** Critical (Rank 1) > Serious (Rank 2) > Minor (Rank 3).
- **Two-Resource Rule:** Admission requires *both* an eligible vacant bed AND an on-duty qualified doctor simultaneously.
- **Referral Policy:** Primary hospital failure immediately triggers referral evaluation; fastest eligible hospital is selected via Dijkstra.
- **Atomic Reservation:** Bed and doctor are locked before referral confirmation.
- **Deterministic Tie-Breaking:** Ties in severity are strictly resolved by arrival sequence (#1, #2, ...).

3. Java OOP Concepts Implementation

Direct mapping to syllabus: Classes/Objects, Encapsulation, Inheritance & Polymorphism

OOP Concept	Syllabus Requirement	Implementation Details in Code
Classes & Objects	Patient, Bed, Doctor, Hospital, AdmissionRequest	• Concrete entities modeling all district domain components. • All classes consolidated cleanly into **HospitalApp.java**.
Encapsulation	Private bed & doctor states change through admitPatient(), dischargePatient(), setDutyStatus()	• **Bed:** private boolean occupied & private String currentPatientId mutate *only* via admitPatient() / dischargePatient(). • **Doctor:** private boolean onDuty & private String assignedPatientId mutate *only* via setDutyStatus(), admitPatient(), and dischargePatient().
Inheritance & Polymorphism	Bed can have GeneralBed and ICUBed subclasses overriding supportsRequirement()	• abstract class Bed defines polymorphic supportsRequirement(String req). • **GeneralBed:** Overrides to match 'General', 'Ward', 'Observation'. • **ICUBed:** Overrides to match 'ICU', 'Ventilator', 'Critical Care'.

4. Abstraction & Composition Architecture

Separation of concerns: Admission eligibility abstraction and multi-level composition

Abstraction: AdmissionPolicy Interface

- **Role:** Decouples admission criteria from hospital storage mechanics.
- **Contract:** boolean isEligible(Hospital h, Patient p).
- **Standard Policy:** Evaluates that hospital has an available bed supporting p.getRequiredFacility() AND an available doctor capable of treating p.getSeverity().
- **Extensibility:** Allows pandemic surge policies or disaster mode criteria without modifying Hospital core code.

Composition: Hospital & ReferralService

- **Hospital Composition:**

Hospital contains collections of Bed and Doctor entities indexed by ID inside HashMaps:

- Map<String, Bed> beds
- Map<String, Doctor> doctors

- **ReferralService Composition:**

ReferralService contains RoutePlanner:

- private final RoutePlanner routePlanner;

Orchestrates capacity checking and Dijkstra shortest-path calculations.

5. Data Structures & Algorithmic Justification

Rigorous mapping to syllabus table with time-complexity justifications

Data Structure	Purpose in Project	Complexity & Technical Justification
PriorityQueue (Min-Heap)	Waiting patients ordered by severity and arrival sequence	**O(log N)** enqueue/dequeue. Guarantees Critical patients preempt Serious/Minor patients. Natural ordering governed by Patient.compareTo().
HashMap (Key-Value)	Patients, beds, doctors, and hospitals by ID	**O(1)** average time complexity for real-time ID queries, bed reservations, doctor lookups, and patient status updates.
Weighted Graph (Adjacency Map)	Hospital and road connections with travel times	Represented as Map<String, Map<String, Integer>> using Java Collections. Direct bidirectional road travel times in minutes. Supports dynamic manual travel-time updates.
ArrayList (Dynamic Array)	Admission, discharge, and referral history	**O(1)** amortized append time. Preserves chronological audit trail of all transactions (Admissions, Referrals, Discharges) stored as AdmissionRequest records.

6. Dijkstra's Algorithm + Min-Heap Routing

Shortest travel-time calculation and multi-stage referral filtering

The Algorithm: RoutePlanner

- **Min-Heap Implementation:** Utilizes PriorityQueue<Map.Entry<String, Integer>> comparing distance values.
- **Time Complexity:** O((V + E) log V), ideal for regional hospital graph networks.
- **Relaxation Step:** Iterates neighbors; updates shortest travel times and predecessor nodes to reconstruct the complete route path.
- **Nalam District Road Network:**
  - **NDGH** (Primary Central Hospital)
  - **VSDH:** 14 mins travel time
  - **TCHC:** 19 mins travel time
  - **KTH:** 30 mins travel time

Referral Pipeline: ReferralService

Step 1: Facility & Capacity Pre-Filtering
Examine referral hospitals. Filter candidates having an available bed satisfying supportsRequirement() AND an available doctor.

Step 2: Dijkstra Shortest-Path Execution
Run RoutePlanner.calculateShortestPaths() from primary hospital location.

Step 3: Fastest Candidate Selection
Identify the reachable candidate with minimum travel time (e.g. VSDH @ 14 mins).

Step 4: Atomic Resource Locking
Immediately admit patient and reserve the referral bed before returning confirmation.

7. Deterministic Ordering & Dynamic Re-Ranking

Syllabus Rule: Deterministic priority order and heap re-adjustment upon field mutation

Deterministic Priority Tie-Breaking

In emergency triage, arbitrary tie-breaking is ethically unacceptable and non-deterministic.

3-Tier Comparison in Patient.compareTo():

1. **Clinical Urgency:** Integer severity rank (1=Critical, 2=Serious, 3=Minor).
2. **Arrival Sequence:** Monotonically increasing counter (e.g., #10 arrives before #11). Earlier arrival wins tie.
3. **Patient ID:** Lexicographical tie-breaker (e.g. 'P-101' < 'P-102') ensuring 100% mathematical determinism across all JVM runs.

Dynamic Priority Re-Ranking

Rubric Requirement:

'Remove and reinsert queued records when their ranking fields change.'

Heap Pitfall in Java:

Mutating an object's field while it is inside a PriorityQueue causes heap invariant corruption because Java's heap does not auto-reindex in-place.

Our Robust Implementation in updatePatientSeverity():

- Step 1: waitingQueue.remove(patient); (Extract from heap)
- Step 2: patient.setSeverity(newSeverity); (Mutate rank)
- Step 3: waitingQueue.offer(patient); (Re-insert & re-heapify)

8. Mandatory Verification Test Deliverables

Automated verification of the 4 failure & contention cases specified in syllabus item 4

Test Case (from Rubric)	Scenario & Input Conditions	System Behavior & Verification Result	Status
1. No ICU Bed	Primary hospital NDGH has 0 available ICU beds. Critical patient arrives requiring ICU.	ReferralService triggers Dijkstra. Selects fastest reachable hospital (**VSDH @ 14 mins**). Referral bed reserved immediately.	**PASS**
2. No Matching Doctor	Free ICU bed exists, but all Critical Care doctors are off-duty.	Admission is blocked. Patient queued in WAITING. When doctor toggles ON-DUTY, patient is **immediately admitted**.	**PASS**
3. Simultaneous Demand for 1 Bed	Only 1 General bed remains. Minor patient arrives first (#1); Serious patient arrives second (#2).	PriorityQueue evaluates clinical urgency first. **Serious patient admitted**; Minor patient remains safely waiting.	**PASS**
4. Discharge Enabling Admission	All beds occupied. Patient waiting in triage queue.	Discharging occupant frees bed and **immediately auto-triggers waiting queue processing**, admitting the waiting patient.	**PASS**

9. System Limitations & Failure Case Handling

Edge cases, network resilience, and real-world clinical operational boundaries

Failure Cases Handled Gracefully

- **Total District Saturation:** When all primary and referral beds/doctors are full, patients remain safely intact in PriorityQueue without data loss or memory leaks.
- **Unreachable Graph Nodes:** If road damage isolates a hospital, Dijkstra returns Integer.MAX_VALUE; ReferralService skips unreachable nodes and evaluates alternate routes.
- **Invalid State Transitions:** IllegalStateException thrown if attempting to discharge an already vacant bed or attend multiple patients simultaneously.

Current Limitations & Future Enhancements

- **Static Travel Times:** Travel times between hospitals are currently fixed or manually updated. *Future:* Integrate Google Maps Distance Matrix API for real-time live traffic congestion.
- **IoT Integration:** *Future:* Connect smart bed telemetry sensors to auto-update occupancy upon patient weight detection.
- **Multi-Hospital Concurrency:** *Future:* Implement Java ConcurrentHashMap and ReentrantLock for high-throughput distributed ambulances.

10. Conclusion & Deliverables Summary

Complete alignment with expected deliverables and submission checklist

Expected Deliverable	Project Implementation & Artifacts	Status
Working Java Application	Single consolidated source file **HospitalApp.java** running in-memory with zero compilation errors.	**COMPLETE**
Strict OOP Requirements	Patient, Bed, Doctor, Hospital, AdmissionRequest with Encapsulation, Polymorphism & Abstraction.	**COMPLETE**
DSA Requirements	PriorityQueue, HashMap, Weighted Graph, ArrayList, Dijkstra's algorithm + Min-Heap.	**COMPLETE**
Documented Test Suite	Automated verification covering No ICU Bed, No Matching Doctor, Contention, and Discharge.	**100% PASS**
Clean Workspace	Single source file with batch launchers (run.bat, test.bat). Zero stale class files.	**VERIFIED**

Thank you! Ready for demonstration and Q&A.;